

M1 Imageurs, Poitiers 2025

Introduction aux macros ImageJ

Fabrice P. Cordelières

5 mai 2025

*Matériel de cours
disponible sur GitHub*



Corrigés en pièces jointes au pdf

TABLE DES MATIÈRES

1 Les bases du langage macro ImageJ	1	Par où commencer?	4
L'outil de base	1	Les étapes du protocole de traitement et d'analyse	4
Le Macro Recorder	1	Identifier les instructions clés à utiliser	4
Éditer une séquence et la sauvegarder : .txt ou .ijm?	1	Générer les vignettes pour la quantification	5
Exécuter une macro ou la stopper .	1	Tester la macro	5
Les fonctions intégrées	1	Gérer les exceptions utilisateur . .	5
Structure des fonctions	1		
Trois logiques différentes	2	3 Fonctionnalisation de la macro	6
Éléments de syntaxe	2	Groupes fonctionnels	6
Trouver ses marques avec ce nouveau langage	2	Utilité des fonctions	6
Commenter son code : quelles balises?	2	Syntaxe des fonctions	6
Les variables : types, opérateurs et portée	2	Différents types de fonctions	6
Les boucles	3	Fonctionnalisation de la macro . .	6
Structure de choix	3	Créer la routine d'analyse	7
2 Premiers pas avec le langage macro ImageJ	4	4 Amélioration de l'expérience utilisateur	8
		Créer une interface utilisateur	8

CHAPITRE 1: LES BASES DU LANGAGE MACRO IMAGEJ



LE PREMIER CHAPITRE EST DESTINÉ à vous montrer les points communs, mais surtout les différences entre les langages de programmation avancés que vous pouvez connaître et le langage de scripting propre à ImageJ. Le langage macro, quoique moins puissant et moins formel qu'un vrai langage de programmation, permet une automatisation rapide de tâches, soit pour une utilisation directe, soit pour le prototypage de tâches complexes qui pourront faire ultérieurement l'objet de modules spécifiques, codés en Java.

L'OUTIL DE BASE

LE MACRO RECORDER

Le Macro Recorder permet d'enregistrer (presque) toutes les opérations réalisées par l'utilisateur. Il suffit de le lancer pour voir se créer une série d'instructions permettant de reproduire le protocole de traitement et d'analyse sur une autre image. Il se trouve dans le menu `Plugins>Macros>Record` et fonctionne de la même manière sous ImageJ et Fiji.

ÉDITER UNE SÉQUENCE ET LA SAUVEGARDER : .TXT OU .IJM ?

Bien que permettant l'édition des instructions, le Macro Recorder n'est pas l'éditeur à proprement parler des macros ImageJ. Pour le faire apparaître, il suffit de cliquer sur le bouton `Create`.

Un éditeur plus complet s'affiche, différent suivant qu'on utilise ImageJ ou Fiji. Sous ImageJ, l'éditeur est un simple éditeur de texte, sans coloration syntaxique. En revanche, il embarque quelques fonctionnalités intéressantes :

- Un outil de débogage, accessible via le menu `Debug`. Il permet notamment l'exécution pas-à-pas de la macro.
- Le `Function Finder`, accessible via le menu `Macros>Functions Finder...` : il permet de visualiser et de réaliser une recherche dans l'ensemble des fonctions macros. Le menu `Help>Macro Functions...` renvoie vers la **page de référence du site d'ImageJ**.

Sous Fiji, l'éditeur de macros offre une coloration syntaxique. En revanche, le `Function Finder` n'est pas disponible : il est remplacé par l'auto-complétion et une aide contextuelle. Les outils de débogage/d'exécution pas-à-pas ne sont pas

disponibles. Une astuce permet néanmoins de faire apparaître l'éditeur de macros ImageJ sous Fiji : il suffit pour d'ouvrir le fichier macro après en avoir ôté l'extension.

Justement, quelle extension pour nos fichiers macros ? Deux conventions sont possibles : au final, le fichier est un simple fichier texte. Peu importe, on pourra utiliser au choix `txt` ou `ijm` (Image J Macro).

EXÉCUTER UNE MACRO OU LA STOPPER

Sous ImageJ, pour exécuter une macro, depuis la fenêtre de l'éditeur, aller dans le menu `Macros>Run Macro` (raccourci clavier `CTRL` ou `CMD+R`). Pour en stopper l'exécution, presser la touche `Esc`.

Pour exécuter une macro sous Fiji, utiliser le bouton `Run` en bas de la fenêtre de l'éditeur ou le menu `Macros>Run Macro` (raccourci clavier `CTRL` ou `CMD+R`). Pour en stopper l'exécution, presser la touche `Esc` ou le bouton `Kill` en bas de la fenêtre de l'éditeur.

LES FONCTIONS INTÉGRÉES

STRUCTURE DES FONCTIONS

Plusieurs structures de commandes co-existent :

- `run("Nom_De_Commande", "Arguments")` : mot-clé `run` prenant 2 arguments 1-le nom de la fonction, 2-une chaîne de caractères contenant les arguments, séparés par des espaces ;
- `nomDeCommande("Nom_De_Commande")` : nom de la fonction, suivi d'une chaîne de caractères contenant les arguments.

Ce ne sont que 2 des formes des commandes macros : il n'y a pas de consensus sur la structure des fonctions. La raison est historique : les fonctions en `"run"` correspondent aux premières fonctions implémentées à l'origine du logiciel, les fonctions plus récentes reprenant une syntaxe plus proche du Java.

TROIS LOGIQUES DIFFÉRENTES

Les instructions peuvent adopter trois types de fonctionnement différents qu'il est important de distinguer :

- **Les méthodes** : ce sont des fonctions qui exécutent une opération ex : redimensionner une image, ajouter une coupe à une pile etc ;
- **Les fonctions** : elles renvoient un résultat qui peut être stocké dans une variable ;
- **Les méthodes de collecte** : c'est le cas par exemple de la fonction `getStatistics(area, mean, min, max, std, histogram)` dont l'appel stocke dans les variables fournies en arguments les informations demandées.

NB : Tout comme en Java, le `;` est obligatoire, utilisé pour marquer la fin d'une instruction.

ÉLÉMENTS DE SYNTAXE

TROUVER SES MARQUES AVEC CE NOUVEAU LANGAGE

A l'évidence, pour des tâches simples, le Macro Recorder peut suffire. En revanche, dès lors qu'il faudra modifier dynamiquement un paramètre, boucler un processus sur un ensemble d'éléments ou adapter le comportement de la macro en fonction d'une entrée, il deviendra nécessaire de connaître la syntaxe d'un ensemble d'éléments de base que sont les variables, les boucles et les structures de choix. Comme dans tout langage de programmation, il sera également important de documenter son code : il est nécessaire de connaître les balises à utiliser.

COMMENTER SON CODE : QUELLES BALISES ?

Les balises de commentaires sont identiques aux balises utilisées en Java :

- **Les commentaires courts** : ils sont délimités par une seule balise, le `//` ;
- **Les commentaires longs** : la balise ouvrante est `/*`, la balise fermante est `*/`.

LES VARIABLES : TYPES, OPÉRATEURS ET PORTÉE

TYPES DE VARIABLES

Dans le langage Macros ImageJ, les variables ne sont pas typées. En revanche, trois catégories de variables sont utilisables :

- **Les variables simples** : elles sont déclarées par leur nom. L'attribution d'une valeur se fait au moyen du signe égal. Les guillemets permettent de définir la nature "chaîne de caractère" du contenu. ex : `maVariable=3`;

ou

```
monAutreVariable="chaîne de caractères";
```

- **Les variables de type tableau** : elles sont déclarées par leur nom et l'attribution (=) suivi du mot-clé `newArray()`. Cette instruction peut prendre trois types d'arguments (voir ci-après). *Attention* : le langage *Macros ImageJ* ne permet pas de créer de tableaux multi-dimensionnels.
 - `monTableau=newArray()` ; : le tableau est initialisé mais n'a pas de dimension ;
 - `monTableau=newArray(n)` ; : le tableau est initialisé sans contenu mais à une dimension de "n" cases ;
 - `monTableau=newArray(1, 2, 3)` ; : le tableau est initialisé et dimensionné en fonction du contenu. Ses cases sont initialisées telles que `monTableau[0]` contient la valeur 1, `monTableau[1]` contient la valeur 2 etc. On peut déterminer la dimension du tableau en ayant recours à l'instruction `monTableau.length`.
- **La liste de couples clé-valeur** : On peut utiliser une unique liste de clés-valeurs. Voici une liste des fonctions les plus utiles :
 - `List.clear()` : efface le contenu de la liste actuelle ;
 - `List.size` : renvoie le nombre d'éléments dans la liste actuelle ;
 - `List.set(cle, valeur)` : ajoute ou modifie la paire clé-valeur dans la liste ;
 - `List.get(cle)` : renvoie la valeur associée à la clé en tant que chaîne de caractères, ou une chaîne vide si la clé n'existe pas dans la liste ;
 - `List.getValue(cle)` : renvoie la valeur numérique associée à la clé. Si elle est absente de la liste ou n'est pas une valeur numérique, l'exécution de la macro est stoppée.

OPÉRATEURS

Les opérateurs applicables aux variables sont résumés dans le tableau pages suivante. Certaines opérations entre variables de nature différentes aboutissent à un transtypage implicite en chaînes de caractères au moment où l'opération est réalisée.

```
myVariable1 = 1
myVariable2 = " variable de type chaîne"
myVariable3 = myVariable1 + myVariable2
print(myVariable3)
L'exécution de la macro renvoie "1 variable de type chaîne"
dans la fenêtre Log
```


Opérateur	Priorité	Description
++	1	Pré ou post incrément
--	1	Pré ou post décrément
-	1	Soustraction bit à bit
!	1	Complémentaire du booléen
~	1	Bit complémentaire
*	2	Multiplication
/	2	Division
%	2	Reste entier de la division
	2	Ou logique bitwise (OR, opération sur les bits)
^	2	Ou exclusif logique (XOR, opération sur les bits)
<<, >>	2	Rotation de bit (vers la gauche ou la droite)
+	3	Addition arithmétique ou concaténation de chaînes de caractères
-	3	Soustraction arithmétique
<, <=	4	Comparaisons : plus petit que, plus petit ou égal à
>, >=	4	Comparaisons : plus grand que, plus grand ou égal à
==, !=	4	Comparaisons : égal à, différent de
&&	5	Et logique (AND, opération sur les booléens)
	5	Ou logique bitwise (OR, opération sur les booléens)
=	6	Attribution
+=, -=, *=, /=	6	Attribution combinée à une opération

PORTÉE DES VARIABLES

Le langage Macros ImageJ permet de créer ses propres groupes fonctionnels (voir le Chapitre 3 - Groupes fonctionnels). Cette possibilité permet de mieux organiser son code et d'en réutiliser certaines parties dans plusieurs macros sans pour autant avoir à tout re-coder. Ces groupes peuvent nécessiter la déclaration de variables en leur coeur : leur contenu ne sera alors pas disponible à l'extérieur de la fonction à moins qu'un retour explicite soit implémenté. De même, une variable déclarée en dehors de la fonction définie par l'utilisateur n'y sera pas directement utilisable : il faudra alors passer son contenu en argument, au moment de l'appel.

Une autre stratégie est envisageable : modifier la portée des variables. L'utilisation du mot-clé `var` devant une variable placée dans le corps de la macro (plutôt en en-tête, pour plus de lisibilité) permet de la transformer en **variable globale**, accessible depuis n'importe quel point du code.

LES BOUCLES

BOUCLE DÉFINIE À PRIORI : FOR

Sa structure est identique à celle utilisée en Java, à la différence qu'il n'est pas possible d'utiliser un itérateur sur une liste :

```
for(initialisation; condition de poursuite; itération) {
    //Instructions
}
```

BOUCLE INDÉFINIE À PRIORI : WHILE

Sa structure est identique à celle utilisée en Java :

```
while(condition) {
    //Instructions
}
```

BOUCLE INDÉFINIE À POSTERIORI : DO/WHILE

Cette boucle est exécutée au moins une fois, l'évaluation de la condition s'effectuant à la fin du bloc. Il se déclare comme suit :

```
do {
    //Instructions
} while (condition);
```

STRUCTURE DE CHOIX

Une seule structure de choix est disponible : la structure `if/then/else - else if` :

```
if(condition1) {
    //Instructions 1
}else{
    //Instructions 2
}else if(condition2){
    //Instructions 3
}
```

NB : Inutile d'essayer, la structure `switch/case` n'est malheureusement pas implémentée...

CHAPITRE 2: PREMIERS PAS AVEC LE LANGAGE MACRO IMAGEJ



U COURS DE CETTE PREMIÈRE SÉQUENCE nous allons explorer le langage macro et découvrir les outils à disposition pour résoudre la problématique posée (voir l'encart "problématique").

PAR OÙ COMMENCER ?

Ayant pris connaissance de la problématique, deux étapes devront être réalisées :

1. Identifier les étapes du protocole de traitement et d'analyse ;
2. Identifier les instructions clés à utiliser.

LES ÉTAPES DU PROTOCOLE DE TRAITEMENT ET D'ANALYSE

L'utilisateur a fourni un protocole détaillé. Le plus simple est donc de reprendre chaque étape proposée et d'initier la macro en utilisant chaque item comme un commentaire. L'espace laissé libre entre chaque commentaire sera progressivement complété avec les instructions adéquates.

ÉTAPE 01 : À VOUS DE JOUER !

A faire :

- Créer une nouvelle macro : Plugins>New>Macro
- En utilisant la syntaxe décrite au "Chapitre 1 - Commenter son code", créer un commentaire par étape de traitement

IDENTIFIER LES INSTRUCTIONS CLÉS À UTILISER

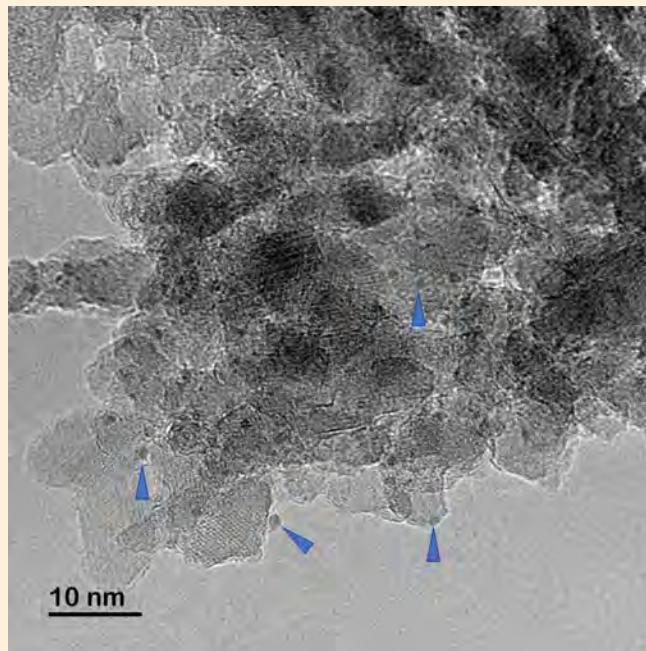
ImageJ dispose d'une fonction permettant d'enregistrement des opérations réalisées sous la forme d'un script : c'est le Macro Recorder, activé au moyen du menu

Plugins>Macros>Record....

Certaines fonctions ne sont pas enregistrables : en complément au Macro Recorder il existe une *page de référence* listant l'ensemble des fonctions accessibles en scripting. En outre, nombre de fonctions documentées incluent des exemples d'utilisation.

D'ores et déjà, au moment de l'enregistrement des instructions, il est important d'anticiper deux points en se posant deux questions. *De quelle situation partons-nous ?* En d'autres termes, une analyse précédente aura-t-elle une

LA PROBLÉMATIQUE



Besoin : L'utilisatrice dispose d'images de microscopie électronique. Elle souhaite, sur un fichier :

- sélectionner manuellement les structures d'intérêt en les pointant avec l'outil de sélection approprié
- pour chaque structure d'intérêt, centrer un carré sur les coordonnées pointées, et créer un duplicata de la zone
- pour chaque vignette, extraire un profil d'intensité horizontal, centré sur la coordonnée pointée
- présenter l'ensemble des profils d'intensités sous la forme d'un graph
- créer un graph de l'intensité moyenne à partir de l'ensemble des profils

incidence sur la nouvelle analyse, en laissant, par exemple, des éléments pouvant compromettre l'analyse courante. *L'analyse actuelle est-elle généralisable ?* Les dénominations des éléments manipulés sont-elles fixes ? Que se passe-t-il si le nom de l'image est différent ? Il est important de prêter attention aux arguments des fonctions utilisées et à la manière dont on pourra les manipuler ultérieurement.

GÉNÉRER LES VIGNETTES POUR LA QUANTIFICATION

On peut se fixer pour première étape de demander à l'utilisateur de pointer les structures d'intérêt et de l'ajouter au ROI Manager. La première partie de la macro aura pour objectif, à partir des points sélectionnés, de créer un carré de taille définie autour du point, puis de dupliquer la portion d'image contenue dans cette région carrée. L'ensemble des images ainsi générées pourra être assemblé en une pile d'images.

ÉTAPE 02 : À VOUS DE JOUER !

A faire :

- Activer le Macro Recorder
Plugins>Macros>Record...
- Réaliser manuellement les étapes de traitement suivantes :
 1. Le cas échéant, vider le ROI Manager.
 2. Ouvrir une image exemple : File>Open...
 3. Activer l'outil de sélection multi-point.
 4. Le cas échéant, éliminer de l'image toute ROI déjà présente.
 5. Demander à l'utilisateur de dessiner une région d'intérêt sur l'image. Attention, cette instruction n'est pas enregistrable : il faudra identifier l'instruction à l'aide de la page de référence.
 6. Ajouter la région au ROI Manager
Edit>Selection>Add to Manager.
 7. Une fois la région multi-point dans le ROI Manager, l'activer, puis extraire les coordonnées XY des points qui la constituent. Attention, cette instruction n'est pas enregistrable : il faudra identifier l'instruction à l'aide de la page de référence.
 8. Définir une variable "size" : elle contiendra la taille du carré à centrer autour de chacun des points de la sélection.
 9. Pour chacun des jeux de coordonnées :
 - (a) Créer une sélection rectangulaire centrée sur la coordonnée XY courante et de côté "size".
 - (b) Dupliquer la portion d'image contenant la région et la nommer "Particle_ZZ" (où ZZ est un index unique associé à la particule courante) :
Image>Duplicate...
 10. Une fois toutes les vignettes générées, les assembler en une seule pile d'images nommée "Thumbnails" (Image>Stacks>Images to Stack...).

TESTER LA MACRO

La première version de la macro étant prête, il est possible de la tester en la lançant depuis

l'éditeur, soit au moyen du bouton Run soit du menu Run>Run. A noter, il est possible de ne sélectionner qu'une partie du code et de n'exécuter que les lignes concernées au moyen du menu Run>Run selected code.

En cas de problème, quelques points à vérifier :

- Manque-t-il un point-virgule pour terminer la ligne précédant la ligne donnée en erreur ?
- Le nom des éléments à manipuler correspondent-ils aux éléments effectivement présents à l'écran ? Vérifier, par exemple, le nom des images à l'écran et les confronter aux noms des images attendus par la macro
- Pour les tables de résultats et le ROI Manager, en cas d'éléments surnuméraires : dans quel état était la ResultsTable/ROI Manager au démarrage de la macro ?
- L'éditeur de macro permet de vérifier les parenthèses, accolades et crochets : chaque balise ouvrante est-elle accompagnée d'une balise fermante ?
- L'instruction qui pose problème est-elle bien orthographiée ? On peut vérifier en enregistrant l'instruction ou en se référant à *cette page*.
- La syntaxe de l'instruction qui pose problème est-elle correcte ? Manque-t-il des arguments ? La nature des arguments est-elle la bonne ? On peut vérifier en enregistrant l'instruction ou en se référant à *cette page*.
- De quelle couleur apparaît l'instruction ? Les arguments ? L'éditeur utilise une coloration basée sur la syntaxe : les commentaires apparaissent en vert, les instructions en orange, les chaînes de caractères en rose et les nombres en violet. Si la couleur n'est pas la couleur attendue, il y a sans doute un problème.

GÉRER LES EXCEPTIONS UTILISATEUR

Et si l'utilisateur avait omis de dessiner une ROI au moment où cela lui est demandé ? La macro planterait au moment d'ajouter la ROI absente au ROI Manager. Comment gérer cette exception ? L'un des moyen serait de vérifier la présence d'une ROI avant de l'ajouter. On peut également envisager d'afficher le message d'invite tant qu'aucune ROI n'est présente sur l'image.

ÉTAPE 03 : À VOUS DE JOUER !

A faire :

- En vous référant au chapitre 1, trouver l'instruction qui permettra d'afficher le message si aucune image n'a été ouverte.
- En vous aidant de *cette page*, trouver l'instruction permettant de vérifier qu'une ROI a été dessinée (renvoie un code correspondant à la nature de la ROI ou -1 si aucune n'est dessinée).
- En amont de l'invite, on peut également faire en sorte d'effacer toute ROI déjà présente sur l'image. L'instruction est enregistrable au travers du menu Edit>Selection>Select None.

CHAPITRE 3: FONCTIONNALISATION DE LA MACRO



LE MOT-CLÉ DE LA SECONDE SÉQUENCE sera "function". Nous allons tirer profit des possibilités de fonctionnalisation qu'offre le code macro. Au-delà du gain en lisibilité du code, la création de fonctions personnalisées permettra une réutilisation simplifiée de pans entiers d'instructions entre macros.

GROUPE FONCTIONNELS

UTILITÉ DES FONCTIONS

Les groupes fonctionnels sont ce que les chapitres sont à un livre : une manière d'organiser les instructions par thématique. Le corps du code apparaît alors comme une table des matières : elle référence l'ensemble des chapitres et permet un accès direct aux sections d'intérêt.

Pour un langage aussi simple que le langage macro ImageJ, outre leur avantage structurant, les fonctions personnalisées généralistes sont utiles dans le cadre d'une réutilisation de blocs, diminuant le temps nécessaire au développement, pourvu qu'elles aient été bien réfléchies en amont.

SYNTAXE DES FONCTIONS

Une fonction est déclarée par le mot-clé `function` suivi du nom de la fonction. Ce nom doit être suffisamment explicite. Par convention, on utilise la notation chameau employée en Java (les mots sont collés les uns aux autres, leur première lettre est capitalisée à l'exception de la lettre initiale). Il doit obligatoirement commencer par une lettre mais peut comporter des chiffres.

Le nom est suivi de deux parenthèses, ouvrante et fermante. Entre les parenthèses peuvent être déclarés des **arguments**, facultatifs : une ou plusieurs variables sont alors fournies entre parenthèses. La portée de ces variables est locale : elles ne sont utilisables qu'à l'intérieur de la fonction. Les **instructions** définissant les opérations réalisées par la fonction sont placées entre **accolades**. Une fonction particulière, facultative, peut également y être présente : le mot-clé `return` permet de renvoyer le contenu d'une unique variable à l'extérieur de la fonction suite à son appel.

L'appel à la fonction se fait simplement en utilisant son nom et fournissant les arguments nécessaires. Si la fonction renvoie un résultat,

il peut être attribué à une variable pour utilisation ultérieure.

```
//Appel à la fonction
resultat=maFonction(1, "arg2");
//Déclaration de la fonction
function maFonction(arg1, arg2, ...){
//Instruction 1
//Instruction 2
...
return valeurDeSortie;
}
```

DIFFÉRENTS TYPES DE FONCTIONS

Les fonctions disposent de deux lots d'éléments facultatifs : les arguments et le retour. Sur la base de ce dernier, on distinguera deux grands types de fonctions : les **méthodes** qui ne disposent pas de l'élément "return" (le retour est `null`) et les **fonctions** qui l'utilisent pour renvoyer une unique variable.

Attention : le fait que le mot-clé ne renvoie le contenu que d'une variable ne signifie pas pour autant qu'on ne puisse renvoyer qu'une valeur. On peut, par exemple utiliser une variable de type tableau pour renvoyer un ensemble d'éléments.

FONCTIONNALISATION DE LA MACRO

Le code de l'**étape 03** réalise la préparation des images pour l'analyse. Il peut être empaqueté au sein d'un groupe fonctionnel simple, ne prenant aucun argument et ne renvoyant aucun résultat.

ÉTAPE 04 : À VOUS DE JOUER!

A faire : Adapter le code de manière à fonctionnaliser la préparation des images :

1. Créer la fonction `generateThumbnails()`.
2. S'interroger sur les lignes de codes à inclure au sein de la fonction et celles à maintenir à l'extérieur.
3. Le cas échéant, réfléchir aux arguments nécessaires pour que la fonction puisse réaliser l'action.
4. Testez la macro.
5. NB : pour qu'une fonction réalise l'opération souhaitée, il faut l'appeler...

CRÉER LA ROUTINE D'ANALYSE

Nous disposons d'une première version de la macro qui, à partir de points entrés par l'utilisateur, permet d'extraire un ensemble de vignettes centrées. Ces images sont stockées au sein d'une pile d'image.

A présent, il faut créer un graph constitué de plusieurs trace. Chaque trace doit correspondre à un profil d'intensité le long d'un axe horizontal passant par le centre de l'image. Ce processus peut être décomposé en une étape élémentaire, qu'il conviendra de répéter pour l'ensemble des coupes de la pile.

ÉTAPE 05 : À VOUS DE JOUER !

A faire : créer une nouvelle macro permettant de placer un axe horizontal passant par le centre de l'image, de rapatrier son profil d'intensité et de le tracer.

1. Les valeurs d'intensités sont à tracer en fonction de la position du pixel en abscisse de l'image. Pour que l'axe des x soit calibré en microns et non en pixels, il conviendra de trouver une fonction permettant de rapatrier la calibration de l'image.
2. Créer une variable de type tableau, nommée x, et remplir chaque cellule avec la distance calibrée du pixel au bord gauche de l'image.
3. Initialiser un graph ayant pour titre "Intensity over an horizontal mid-line", pour légende des abscisses "Position in x (unité)", et pour légende des ordonnées "Intensity (AU)".
4. Positionner une sélection de type ligne horizontalement, passant par le centre de l'image.
5. Rapatrier les valeurs d'intensités le long de la ligne et les stocker dans une variable de type tableau, nommée "intensityProfile".
6. Ajouter au graph précédent le tracé de type "line", à partir des données (x, intensityProfile).
7. Afficher le graph au moyen de l'instruction `Plot.show();`.
8. NB : En amont de l'ajout du tracé, on peut modifier sa future couleur d'affichage au moyen de l'instruction `Plot.setColor(color)`.
9. NB2 : En amont de la modification de couleur, on peut définir une série de couleurs à utiliser au moyen d'une variable de type tableau :

```
colors=newArray("black", "blue",  
"cyan", "darkGray", "gray", "green",  
"lightGray", "magenta", "orange",  
"pink", "red", "white", "yellow");
```

Nous avons maintenant le processus pour l'image courante de la pile : reste à répéter l'opération sur l'ensemble des coupes, en veillant à changer d'image après chaque ajout de tracé sur le graph et en prenant garde à bien définir sur quelle image l'analyse est réalisée (le graph est aussi une image).

ÉTAPE 06 : À VOUS DE JOUER !

A faire : adapter la macro précédente afin de répéter le processus pour l'ensemble des images de la pile.

- Pour toutes les images de la pile :
 - Activer la i-ème image de la pile.
 - Appliquer l'algorithme de création de la ligne et ajout du graph, en veillant à modifier la couleur du tracé à chaque ajout au graph.

L'utilisatrice souhaite qu'un second graph soit créé, reprenant la moyenne des profils d'intensités. Une fois cette fonctionnalité ajoutée, on pourra empaqueter l'ensemble du code sous la forme d'une nouvelle fonction.

ÉTAPE 07 : À VOUS DE JOUER !

A faire : adapter la macro précédente afin d'ajouter la génération du graph moyen. Créer un groupe fonctionnel permettant de réaliser l'ensemble de l'extraction des données et de la création des graphs.

1. Créer la fonction `getIntensityProfiles`. Elle réalise les opérations suivantes :
2. Pour chaque image de la pile :
 - Elle réalise un tracé du profil d'intensité, chaque tracé ayant une couleur différente.
 - Elle garde l'information des tracés pour calculer ultérieurement une moyenne.
3. Elle crée un graph présentant la moyenne des profils d'intensités
4. Testez cette nouvelle fonction.
5. Intégrez cette fonction au corps de la macro de l'étape 04
 - NB : pour qu'une fonction réalise l'opération souhaitée, il faut l'appeler...

CHAPITRE 4: AMÉLIORATION DE L'EXPÉRIENCE UTILISATEUR

POUR CETTE DERNIÈRE SÉQUENCE, NOUS allons améliorer l'expérience utilisateur. Nous avons implémenté une analyse qui requiert un certain nombre de paramètres pour être réalisée : une boîte de dialogue pour les entrer plutôt que d'avoir à éditer directement le code.

CRÉER UNE INTERFACE UTILISATEUR

La création d'interface (**Graphical User Interface** ou GUI en anglais), quoique rudimentaire en langage macro, est relativement simple et puissante. Une fois la boîte de dialogue affichée, il faudra rapatrier l'ensemble des entrées utilisateur. Les éléments de la GUI ne sont pas stockés dans des variables indépendantes : on ne les adresse pas directement. Le mécanisme de collection se fait dans l'ordre où les éléments ont été ajoutés à l'affichage, au moyen de getters (fonctions permettant d'obtenir un résultat) spécifiques.

Créer une GUI (source: doc. en ligne) :

Dialog.create("Title")

Creates a modal dialog box with the specified title, or use **Dialog.createNonBlocking("Title")** to create a non-modal dialog. Call **Dialog.addString()**, **Dialog.addNumber()**, etc. to add components to the dialog. Call **Dialog.show()** to display the dialog and **Dialog.getString()**,

Dialog.getNumber(), etc. to retrieve the values entered by the user. Refer to the **DialogDemo** macro for an example.

Dialog.addNumber(label, default) - Adds a numeric field to the dialog, using the specified label and default value.

Dialog.show() - Displays the dialog and waits until the user clicks "OK" or "Cancel". The macro terminates if the user clicks "Cancel".

Dialog.getNumber() - Returns the contents of the next numeric field.

Néanmoins, nous n'avons ici qu'un seul paramètre à obtenir de l'utilisateur : la taille des carrés à placer autour des points d'intérêt. On pourra alors utiliser une instruction simple, permettant d'obtenir le nombre en question.

ÉTAPE 8 : À VOUS DE JOUER !

Finaliser la macro :

- Bien penser à démarrer la macro par une étape de nettoyage/remise à zéro.
- Faciliter l'utilisation de la macro en activant le bon outil de sélection (multi-point tool).
- Inviter l'utilisateur à créer la sélection : il existe une instruction permettant d'"attendre une action de l'utilisateur", mettant en pause l'exécution tant qu'il n'a pas cliqué sur le bouton "Ok" d'une boîte de dialogue dédiée.
- Ajouter la ROI au ROI Manager.
- Demander et stocker la taille des carrés à placer autour des points d'intérêt.
- Appeler tout à tour la fonction de génération des imagerie et celle de tracé des graphs.